

Y E H I O R T E C H

MédiHelm

L'Infrastructure Pharmaceutique Numérique du Bénin

SPECIFICATIONS TECHNIQUES ET FONCTIONNELLES — VERSION 2.0

Modules M16-M19 · API

institutionnelles · DPMED · SoBAPS · UbiPharm · Pharmacovigilance · Conformité RBAC étendu

Référence document

MH-SPECS-2025-v2.0 (remplace MH-SPECS-2025-v1.0)

Document parent

MH-CDC-2025-v2.0 (Cahier des Charges v2.0)

Responsable

Dawes — Fondateur & Lead Developer, YEHI OR Tech

Profil institutionnel

Ancien stagiaire LNCQ — Ministère de la Santé Bénin

Stack

Next.js 14 · NestJS · PostgreSQL · Prisma · Supabase · Vercel · Fedapay

Statut

Spécifications de référence v2.0 — Confidentiel

Document confidentiel — YEHI OR Tech © 2025

Sommaire

1. Architecture v2.0 — Nouveaux composants institutionnels
2. Schéma Prisma v2.0 — Nouveaux modèles
3. RBAC v2.0 — Rôles institutionnels et matrice étendue
4. API REST v2.0 — Endpoints institutionnels
5. Module M16 — Pharmacovigilance : spécifications techniques
6. Module M17 — Intégrations SoBAPS et grossistes : spécifications
7. Module M18 — Alertes DPMED : spécifications et diffusion
8. Module M19 — Conformité réglementaire : spécifications
9. Portail institutionnel — Spécifications frontend
10. Sécurité renforcée — mTLS et données institutionnelles
11. Queue de diffusion d'alertes — Architecture haute priorité
12. Analytics IA v2.0 — Domaines enrichis
13. Tests v2.0 — Parcours institutionnels
14. Déploiement v2.0 — Variables d'environnement étendues

1. Architecture v2.0 — Nouveaux Composants Institutionnels

1.1 Vue d'ensemble enrichie

La v2.0 ajoute une couche institutionnelle à l'architecture existante. Trois nouveaux modules NestJS (dpmed-gateway, grossistes-gateway, sobaps-gateway) gèrent les intégrations avec les acteurs de la chaîne pharmaceutique. Un espace frontend dédié (portail-institutionnel) est servi sur un sous-domaine séparé. Une queue BullMQ haute priorité gère la diffusion des alertes sanitaires.

1.2 Nouveaux composants v2.0

Composant	Technologie	Rôle	URL
Portail institutionnel	Next.js — sous-domaine séparé	Interface DPMED, SoBAPS, ABRP, grossistes	institutionnel.medihelm.com
DPMED Gateway	NestJS module	Réception alertes DPMED + envoi signalements pharmacovigilance	Interne API
Grossistes Gateway	NestJS module	API UbiPharm + Promopharma — commandes et confirmations	Interne API
SoBAPS Gateway	NestJS module	Confirmations livraisons SoBAPS + réconciliation stock	Interne API
Base Veille Qualité	PostgreSQL table dédiée	Médicaments suspects, contrefaits, rappels de lot	Schéma qualite
Queue Alertes DPMED	BullMQ — priorité CRITICAL	Diffusion alertes < 2 min — file dédiée haute priorité	Redis queue:alertes
Webhook DPMED receiver	NestJS controller	Réception alertes en temps réel depuis système DPMED	POST /webhooks/dpmed
Webhook UbiPharm receiver	NestJS controller	Confirmations de commande UbiPharm	POST /webhooks/ubipharm
Webhook Promopharma receiver	NestJS controller	Confirmations de commande Promopharma	POST /webhooks/promopharma
Webhook SoBAPS receiver	NestJS controller	Confirmations livraisons SoBAPS	POST /webhooks/sobaps

1.3 Architecture des schémas PostgreSQL

```
-- Séparation des schémas pour isolation des données

-- Schéma principal (existant v1.0)
CREATE SCHEMA pharma;
-- Contient: pharmacies, utilisateurs, médicaments, ventes,
--          patients, employes, commandes, etc.

-- Schéma institutionnel (nouveau v2.0)
CREATE SCHEMA institutionnel;
-- Contient: alertes_dpmed, partenaires_institutionnels,
--          confirmations_sobaps, portails_accès

-- Schéma qualité (nouveau v2.0)
CREATE SCHEMA qualite;
-- Contient: médicaments_surveillance, signalements_ei,
--          rappels_lot, fiches_dci

-- Schéma grossistes (nouveau v2.0)
CREATE SCHEMA grossistes;
-- Contient: commandes_api, confirmations_livraison,
--          catalogue_prix, partenaires_grossistes

-- RLS active sur tous les schémas
-- Aucune jointure directe entre schéma pharma et institutionnel
```

2. Schéma Prisma v2.0 — Nouveaux Modèles

2.1 Modèles — Module M16 Pharmacovigilance

```
// Schéma: qualite

model MedicamentSurveillance {
  id          String      @id @default(uuid())
  dci         String
  nomCommercial String?
  fabricant   String?
  numeroLot   String?
  typeSurveillance TypeSurv
  description  String
  sourceAlerte String      // DPMED, LNCQ, OMS, ABRP
  dateEmission DateTime
  dateExpiration DateTime? // null = alerte permanente
  niveauRisque NiveauRisque
  statut       StatutAlerte @default(ACTIVE)
  documentUrl  String?
  createdAt    DateTime      @default(now())
  updatedAt    DateTime      @updatedAt
  signalements SignalementEI[]
  alertesEmises AlerteDPMED[]
}

enum TypeSurv { CONTREFACON NON_CONFORME RAPPEL_LOT AMM_SUSPENDUE
SOUS_SURVEILLANCE }
enum NiveauRisque { CRITIQUE ELEVE MODERE INFORMATIF }
enum StatutAlerte { ACTIVE LEVEE ARCHIVEE }

model SignalementEI {
  id          String      @id @default(uuid())
  pharmacieId String
  utilisateur String
  patientCode String      // pseudonymisé - jamais le nom réel
  medicamentId String?
  medicamentSurvId String?
  dciConcernee String
  descriptionEI String
  gravite      GraviteEI
  dateDebut    DateTime
  evolution    String?
  actionPrise  String?
  statutEnvoi  StatutEnvoi @default(EN_ATTENTE)
  refDPMED     String?     // référence attribuée par DPMED
  createdAt    DateTime      @default(now())
  medicamentSurv MedicamentSurveillance?
  @relation(fields:[medicamentSurvId], references:[id])
}

enum GraviteEI { LEGER MODERE GRAVE FATAL }
enum StatutEnvoi { EN_ATTENTE SOUMIS ACCUSE_RECEPTION EN_ANALYSE CLOTURE }

model FicheDCI {
```

```

    id                String      @id @default(uuid())
    dci                String      @unique
    classeThera        String
    mecanismeAction    String
    indicationsPrincipales String
    posologie          String
    contreIndications  String
    interactions        Json       // tableau interactions [{dci, gravite,
description}]
    effetsIndesirables Json       // tableau EI fréquents
    conservation       String
    sourceVersion      String      // WHO EML version
    updatedAt          DateTime    @updatedAt
  }

```

2.2 Modèles — Module M17 Intégrations Grossistes

```

// Schéma: grossistes

model PartenaireGrossiste {
  id                String      @id @default(uuid())
  nom               String      // UbiPharm, Promopharma
  codeGrossiste     String      @unique
  apiEndpoint       String
  apiKeyHash        String      // jamais en clair
  webhookSecret     String      // HMAC signature
  actif             Boolean      @default(true)
  createdAt         DateTime    @default(now())
  commandesAPI      CommandeGrossiste[]
  cataloguePrix     CataloguePrix[]
}

model CommandeGrossiste {
  id                String      @id @default(uuid())
  pharmacieId       String
  grossisteId       String
  commandeInterne   String      // référence CommandeFournisseur interne
  referenceGrossiste String?    // attribuée par le grossiste
  statut            StatutCmdAPI @default(ENVOYEE)
  montantTotal      Float?
  dateEnvoi         DateTime    @default(now())
  dateConfirmation  DateTime?
  dateLivraisonPrev DateTime?
  dateLivraisonReelle DateTime?
  payload           Json       // données complètes de la commande
  reponseGrossiste  Json?
  grossiste         PartenaireGrossiste @relation(fields:[grossisteId],
references:[id])
}

enum StatutCmdAPI { ENVOYEE CONFIRMEE REFUSEE EN_PREPARATION EN_LIVRAISON
LIVREE LITIGE }

```

```

model CataloguePrix {
  id                String      @id @default(uuid())
  grossisteId       String
  referenceGros      String
  dci                String
  nomCommercial      String
  forme              String
  dosage             String
  prixAchat          Float
  disponible         Boolean    @default(true)
  updatedAt          DateTime   @updatedAt
  grossiste          PartenaireGrossiste @relation(fields:[grossisteId],
references:[id])
  @@unique([grossisteId, referenceGros])
}

```

2.3 Modèles — Module M18 Alertes DPMED

```

// Schéma: institutionnel

model AlerteDPMED {
  id                String      @id @default(uuid())
  referenceOfficielle String    @unique // ref DPMED officielle
  titre             String
  description        String
  typeAlerte         TypeAlerteDPMED
  niveauUrgence      NiveauUrgence
  medicamentSurvId   String?
  dciConcernee       String?
  numerosLotConcernees String[]
  fabricantConcerne   String?
  sourceEmission      String    // DPMED, LNCQ, SoBAPS
  documentOfficielUrl String?
  signatureNumerique String    // vérification authenticité
  dateEmissionDPMED   DateTime
  dateReceptionMH     DateTime @default(now())
  statut             StatutAlerteMH @default(EN_DIFFUSION)
  nbOfficinesNotifiees Int      @default(0)
  nbOfficinesAcquittees Int     @default(0)
  nbPatientsNotifies Int       @default(0)
  medicamentSurv      MedicamentSurveillance?
  @relation(fields:[medicamentSurvId], references:[id])
  diffusions          DiffusionAlerte[]
}

enum TypeAlerteDPMED { RAPPEL_LOT CONTREFACON AMM_SUSPENDUE PHARMACOVIGILANCE
INFO_REGLEMENTAIRE }
enum NiveauUrgence { URGENCE_IMMEDIATE URGENT NORMAL INFORMATIF }
enum StatutAlerteMH { EN_DIFFUSION DIFFUSEE ARCHIVEE }

model DiffusionAlerte {
  id                String      @id @default(uuid())
  alerteId           String
  pharmacieId        String
}

```

```

    lotsConcernes    String[]    // lots de cette pharmacie concernés
    canalEnvoi        String[]    // ['PUSH', 'SMS', 'EMAIL']
    dateEnvoi         DateTime    @default(now())
    dateAcquittement  DateTime?
    actionPrise       String?     // quarantaine, destruction, retour grossiste
    alerte            AlerteDPMED @relation(fields:[alerteId], references:[id])
  }

model ConfirmationReceptionSoBAPS {
  id                String        @id @default(uuid())
  pharmacieId       String
  bonLivraisonRef   String        // référence bon de livraison SoBAPS
  dateReception     DateTime    @default(now())
  statut            StatutConf    @default(CONFORME)
  ecarts            Json?        // liste des écarts constatés
  webhookSent       Boolean       @default(false)
  webhookSentAt     DateTime?
}

enum StatutConf { CONFORME AVEC_ECART REFUSE }

```

2.4 Modèles — Module M19 Conformité

```

// Schéma: institutionnel

model ScoreConformite {
  id                String        @id @default(uuid())
  pharmacieId       String        @unique
  scoreTotal        Float         // 0-100
  scoreRegistreStup  Float         // 0-25
  scoreAlerteDPMED   Float         // 0-25
  scoreDocuments     Float         // 0-20
  scorePharmacovigi  Float         // 0-15
  scoreDestructions  Float         // 0-15
  certificationDPMED Boolean       @default(false)
  dateCertification  DateTime?
  dateExpirCertification DateTime?
  calculatedAt       DateTime    @default(now())
}

model DocumentReglementaire {
  id                String        @id @default(uuid())
  pharmacieId       String
  typeDocument      TypeDocReg
  fileUrl           String        // Supabase Storage chiffré
  dateEmission       DateTime?
  dateExpiration     DateTime?
  statut            StatutDoc    @default(VALIDE)
  alertes90j        Boolean       @default(false)
  alertes60j        Boolean       @default(false)
  alertes30j        Boolean       @default(false)
  createdAt         DateTime    @default(now())
}

```

```
enum TypeDocReg { LICENCE_EXPLOITATION DIPLOME_PHARMACIEN ASSURANCE  
AGREMENT_DPMED AUTORISATION_STUPEFIANTS }  
enum StatutDoc { VALIDE EXPIRE_BIENTOT EXPIRE SUSPENDU }
```

3. RBAC v2.0 — Rôles Institutionnels et Matrice Étendue

3.1 Nouveaux rôles institutionnels

Rôle	Périmètre	Création du compte
DPMED_ADMIN	Emission d'alertes, consultation tableau de bord DPMED, gestion base veille qualité	YEHI OR Tech Admin uniquement
SOBAPS_VIEWER	Consultation confirmations réception des livraisons SoBAPS — lecture seule	YEHI OR Tech Admin uniquement
ABRP_VIEWER	Consultation tableau de bord agrégé anonymisé — lecture seule	YEHI OR Tech Admin uniquement
GROSSISTE_PARTNER	Consultation commandes reçues + tableau de bord demande agrégée — lecture seule	YEHI OR Tech Admin uniquement
PLATFORM_ADMIN	Accès total à toute la plateforme — YEHI OR Tech uniquement	Système

3.2 Matrice RBAC étendue — Nouveaux modules

Module	ADMIN	DPMED	SOBAPS	ABRP	GROSSISTE	DIREC	PHARMA	CAISS
M16 — Veille qualité consultation	RWD	RWD	R	-	-	R	R	-
M16 — Signalement pharmacovigilance	RWD	RWD	-	-	-	RW	RW	-
M16 — Gestion base surveillance	RWD	RW	-	-	-	R	R	-
M17 — API grossistes	RWD	-	-	-	R portail	RW	R	-
M17 — Catalogue grossiste	RWD	-	-	-	RW portail	R	R	-
M17 — Confirmations SoBAPS	RWD	-	R portail	-	-	RW	R	-
M18 — Emission alerte DPMED	RWD	RW portail	-	-	-	-	-	-
M18 — Réception alerte	RWD	R	R	-	-	R	R	R
M18 — Acquiescement alerte	RWD	-	-	-	-	RW	R	-
M18 — Tableau de bord DPMED	RWD	R portail	-	-	-	-	-	-

Module	ADMIN	DPMED	SOBAPS	ABRP	GROSSISTE	DIRECT	PHARMA	CAISS
M18 — Tableau de bord SoBAPS	RWD	-	R portail	-	-	-	-	-
M18 — Tableau de bord ABRP	RWD	-	-	R portail	-	-	-	-
M19 — Score conformité	RWD	-	-	-	-	R	R	-
M19 — Documents réglementaires	RWD	-	-	-	-	RW	-	-
M19 — Certification DPMED	RWD	RW	-	-	-	R	-	-

4. API REST v2.0 — Endpoints Institutionnels

4.1 Module M16 — Pharmacovigilance

Méthode	Endpoint	Description	Rôles
GET	/qualite/surveillance	Liste médicaments sous surveillance	DIRECTEUR, PHARMACIEN
GET	/qualite/surveillance/:id	Détail médicament sous surveillance	DIRECTEUR, PHARMACIEN
GET	/qualite/surveillance/check/:dci	Vérifier statut DCI	Tous rôles + Public
POST	/qualite/signalements	Créer un signalement EI	DIRECTEUR, PHARMACIEN
GET	/qualite/signalements	Liste signalements de l'officine	DIRECTEUR, PHARMACIEN
GET	/qualite/signalements/:id	Détail signalement	DIRECTEUR, PHARMACIEN
PATCH	/qualite/signalements/:id	Modifier un signalement	DIRECTEUR, PHARMACIEN
POST	/qualite/signalements/:id/soumettre	Soumettre à la DPMED	DIRECTEUR, PHARMACIEN
GET	/qualite/dci/:dci	Fiche DCI complète	Tous rôles pharmacie
GET	/qualite/interactions	Vérifier interactions entre DCI	DIRECTEUR, PHARMACIEN
GET	/qualite/score	Score contrôle qualité de l'officine	DIRECTEUR

4.2 Module M17 — Intégrations Grossistes et SoBAPS

Méthode	Endpoint	Description	Rôles
GET	/grossistes	Liste des grossistes partenaires	DIRECTEUR+
GET	/grossistes/:id/catalogue	Catalogue produits du grossiste	DIRECTEUR, PHARMACIEN
POST	/grossistes/:id/commandes	Envoyer commande via API grossiste	DIRECTEUR, PHARMACIEN
GET	/grossistes/:id/commandes	Commandes envoyées à ce grossiste	DIRECTEUR
GET	/grossistes/commandes/:id	Statut d'une commande API	DIRECTEUR, PHARMACIEN

Méthode	Endpoint	Description	Rôles
GET	/grossistes/compare/:dci	Comparer prix multi-grossistes	DIRECTEUR, PHARMACIEN
POST	/sobaps/receptions	Enregistrer réception livraison SoBAPS	DIRECTEUR, MAGASINIER
GET	/sobaps/receptions	Historique réceptions SoBAPS	DIRECTEUR, MAGASINIER
GET	/sobaps/receptions/:id	Détail réception + écarts	DIRECTEUR
POST	/webhooks/ubiPharm	Webhook confirmations UbiPharm	Système — signature HMAC
POST	/webhooks/promopharma	Webhook confirmations Promopharma	Système — signature HMAC
POST	/webhooks/sobaps	Webhook confirmations SoBAPS	Système — signature HMAC

4.3 Module M18 — Alertes DPMED

Méthode	Endpoint	Description	Rôles
GET	/alertes/dpmed	Alertes DPMED actives pour l'officine	Tous rôles pharmacie
GET	/alertes/dpmed/:id	Détail d'une alerte	Tous rôles pharmacie
POST	/alertes/dpmed/:id/acquitter	Accuser réception d'une alerte	DIRECTEUR, PHARMACIEN
POST	/alertes/dpmed/:id/action	Documenter l'action prise	DIRECTEUR, PHARMACIEN
GET	/alertes/dpmed/historique	Historique alertes traitées	DIRECTEUR
POST	/webhooks/dpmed	Webhook réception alerte depuis DPMED	Système — signature numérique
GET	/portail/dpmed/alertes	Portail DPMED — toutes alertes émises	DPMED_ADMIN
POST	/portail/dpmed/alertes	Portail DPMED — émettre une alerte	DPMED_ADMIN
GET	/portail/dpmed/dashboard	Tableau de bord DPMED — taux acquittement	DPMED_ADMIN
GET	/portail/sobaps/confirmations	Portail SoBAPS — confirmations réceptions	SOBAPS_VIEWER
GET	/portail/abrp/dashboard	Portail ABRP — agrégats anonymisés	ABRP_VIEWER

Méthod e	Endpoint	Description	Rôles
GET	/portail/grossiste/commandes	Portail grossiste — commandes reçues	GROSSISTE_PARTNER

4.4 Module M19 — Conformité

Méthod e	Endpoint	Description	Rôles
GET	/conformite/score	Score de conformité de l'officine	DIRECTEUR
GET	/conformite/documents	Liste documents réglementaires	DIRECTEUR
POST	/conformite/documents	Uploader un document réglementaire	DIRECTEUR
DELETE	/conformite/documents/:id	Supprimer un document	DIRECTEUR
GET	/conformite/exports/stupefians	Registre stupéfiants — format DPMED	DIRECTEUR, PHARMACIEN
GET	/conformite/exports/ordonnances	Registre ordonnances — format DPMED	DIRECTEUR, PHARMACIEN
GET	/conformite/exports/destructions	PV destructions — format DPMED	DIRECTEUR
GET	/conformite/exports/declaration-trimestrielle	Déclaration trimestrielle stupéfiants	DIRECTEUR
GET	/conformite/exports/pharmacovigilance-mensuel	Rapport PV mensuel — format OMS	DIRECTEUR, PHARMACIEN
GET	/conformite/certification	Statut certification DPMED	DIRECTEUR
POST	/conformite/certification/demander	Initier demande de certification DPMED	DIRECTEUR

5. Module M16 — Pharmacovigilance : Spécifications Techniques

5.1 Vérification qualité lors de la réception de stock

```
// Lors de la réception d'un lot (M03 réception commande)
// Le service de veille qualité est appelé automatiquement

async verifierLotRecu(numeroLot: string, dci: string) {
  const alertes = await this.prisma.medicamentSurveillance.findMany({
    where: {
      statut: 'ACTIVE',
      OR: [
        { dci: { contains: dci, mode: 'insensitive' } },
        { numeroLot: numeroLot }
      ]
    }
  });

  if (alertes.length > 0) {
    // Bloquer la réception + notifier le pharmacien
    await this.notifService.alerteReceptionBloquee({
      pharmacieId, numeroLot, alertes
    });
    throw new ConflictException({
      code: 'MH-QUAL-001',
      message: 'Lot sous surveillance DPMED – réception bloquée',
      alertes
    });
  }
}
```

5.2 Signalement EI — Processus de soumission à la DPMED

```
// Soumission signalement EI à la DPMED
async soumettreSignalement(id: string, pharmacieId: string) {
  const signalement = await this.prisma.signalementEI.findUniqueOrThrow({
    where: { id }, include: { medicamentSurv: true }
  });

  // Construction du payload format DPMED / OMS
  const payload = {
    source: 'MediHelm',
    date: new Date().toISOString(),
    patient: signalement.patientCode, // pseudonymisé
    medicament: signalement.dciConcernee,
    effetIndes: signalement.descriptionEI,
    gravite: signalement.gravite,
    action: signalement.actionPrise,
    pharmacieRef: pharmacieId,
  };
}
```

```
// Si API DPMED disponible
if (this.dpmedGateway.isConnected()) {
  const ref = await this.dpmedGateway.submitSignalement(payload);
  await this.prisma.signalementEI.update({
    where: { id },
    data: { statutEnvoi: 'ACCUSE_RECEPTION', refDPMED: ref }
  });
} else {
  // Fallback : génération PDF format DPMED pour envoi manuel
  const pdf = await this.pdfService.generateSignalement(payload);
  await this.storageService.save(`signalements/${id}.pdf`, pdf);
  await this.prisma.signalementEI.update({
    where: { id },
    data: { statutEnvoi: 'SOUMIS' }
  });
}
}
```

5.3 Vérification d'interactions médicamenteuses

```
// Vérification lors de la délivrance sur ordonnance multi-médicaments
async verifierInteractions(dcis: string[]): Promise<Interaction[]> {
  const interactions: Interaction[] = [];

  for (let i = 0; i < dcis.length; i++) {
    for (let j = i + 1; j < dcis.length; j++) {
      const fiche = await this.prisma.ficheDCI.findUnique({
        where: { dci: dcis[i] }
      });
      if (!fiche) continue;

      const fiches = fiche.interactions as any[];
      const inter = fiches.find(x =>
        x.dci.toLowerCase().includes(dcis[j].toLowerCase())
      );

      if (inter && (inter.gravite === 'MAJEURE' || inter.gravite ===
        'MODEREE')) {
        interactions.push({
          dcil: dcis[i], dci2: dcis[j],
          gravite: inter.gravite,
          description: inter.description
        });
      }
    }
  }
  return interactions; // vide = pas d'interaction connue
}
```

6. Module M17 — Intégrations SoBAPS et Grossistes : Spécifications

6.1 Architecture du Grossistes Gateway

```
// NestJS Module: GrossistesGateway
@Module({
  providers: [
    GrossistesGatewayService,
    UbiPharmAdapter,
    PromoPharmAdapter,
  ]
})
export class GrossistesGatewayModule {}

// Interface commune - chaque adaptateur l'implémente
interface IGrossisteAdapter {
  sendCommande(cmd: CommandeGrossiste): Promise<string>; // retourne ref grossiste
  getConfirmation(ref: string): Promise<ConfirmationGrossiste>;
  getCatalogue(): Promise<ProduitGrossiste[]>;
}

// Si l'API d'un grossiste n'est pas disponible :
// Fallback EDI - email structuré avec le bon de commande en pièce jointe
// Format JSON embarqué dans l'email pour parsing automatique côté grossiste
```

6.2 Transmission d'une commande API

```
// Workflow complet d'une commande via API UbiPharm

// 1. Pharmacien valide la commande depuis M03
// 2. M03 appelle GrossistesGateway si fournisseur = partenaire API
async transmettreCommandeAPI(commandeId: string) {
  const cmd = await this.prisma.commandeFournisseur.findUniqueOrThrow({
    where: { id: commandeId }, include: { lignes: true, fournisseur: true }
  });

  const grossiste = await this.prisma.partenaireGrossiste.findFirst({
    where: { nom: { contains: cmd.fournisseur.nom } }
  });
  if (!grossiste) return; // pas de partenariat API - flux normal

  const adapter = this.getAdapter(grossiste.codeGrossiste);
  const refGrossiste = await adapter.sendCommande({
    pharmacieRef: cmd.pharmacieId,
    lignes: cmd.lignes.map(l => ({
      refGrossiste: l.refGrossiste,
      dci: l.dci,
      quantite: l.quantiteCommandee,
      dateSouhaitee: cmd.dateLivraison
    }))
  });
}
```

```

});

// Enregistrer la commande API avec la référence grossiste
await this.prisma.commandeGrossiste.create({
  data: { pharmacieId: cmd.pharmacieId, grossisteId: grossiste.id,
    commandeInterne: commandeId, referenceGrossiste: refGrossiste,
    statut: 'ENVOYEE', payload: cmd }
});

// Notification pharmacien : commande envoyée automatiquement
this.gateway.emit(cmd.pharmacieId, 'commande:api:envoyee', { commandeId,
refGrossiste });
}

```

6.3 Réception SoBAPS — Réconciliation stock

```

// Réception d'une livraison SoBAPS
async enregistrerReceptionSoBAPS(dto: ReceptionSoBAPSDto) {
  const { pharmacieId, bonLivraisonRef, lignes } = dto;

  // 1. Mettre à jour le stock pour chaque médicament reçu
  for (const ligne of lignes) {
    await this.stockService.ajouterLot({
      pharmacieId,
      medicamentId: ligne.medicamentId,
      numeroLot: ligne.numeroLot,
      quantite: ligne.quantiteReelle,
      prixAchat: ligne.prixSoBAPS,
      dateExpiration: ligne.dateExpiration,
      fournisseur: 'SoBAPS'
    });
  }

  // 2. Calculer les écarts bon de livraison vs reçu
  const ecarts = lignes
    .filter(l => l.quantiteReelle !== l.quantiteBL)
    .map(l => ({ dci: l.dci, attendu: l.quantiteBL, recu: l.quantiteReelle }));

  // 3. Enregistrer la confirmation
  const conf = await this.prisma.confirmationReceptionSoBAPS.create({
    data: { pharmacieId, bonLivraisonRef,
      statut: ecarts.length === 0 ? 'CONFORME' : 'AVEC_ECART',
      ecarts: ecarts.length > 0 ? ecarts : undefined }
  });

  // 4. Envoyer webhook à SoBAPS (si partenariat actif)
  if (this.sobapsGateway.isConnected()) {
    await this.sobapsGateway.confirmReception({
      bonLivraisonRef, statut: conf.statut, ecarts
    });
  }
}

```

7. Module M18 — Alertes DPMED : Architecture de Diffusion

7.1 Queue haute priorité — Architecture BullMQ

```
// Queue dédiée aux alertes - priorité CRITIQUE
const alertesQueue = new Queue('alertes-dpmed', {
  connection: redis,
  defaultJobOptions: {
    priority: 1,           // priorité maximale
    attempts: 5,          // 5 tentatives en cas d'échec
    backoff: { type: 'exponential', delay: 2000 },
    removeOnComplete: false, // garder les logs
    removeOnFail: false
  }
});

// Worker dédié - 20 jobs simultanés (vs 5 pour SMS normaux)
const alertesWorker = new Worker('alertes-dpmed', async job => {
  const { alerteId } = job.data;
  await this.diffuserAlerteComplete(alerteId);
}, { concurrency: 20, connection: redis });
```

7.2 Processus de diffusion d'une alerte DPMED

```
async diffuserAlerteComplete(alerteId: string) {
  const alerte = await this.prisma.alerteDPMED.findUniqueOrThrow({
    where: { id: alerteId }
  });

  // 1. Vérifier la signature numérique de l'alerte
  const valid = await this.verifierSignature(alerte.signatureNumerique, alerte);
  if (!valid) throw new Error('MH-SEC-001: Signature alerte invalide - diffusion annulée');

  // 2. Identifier les pharmacies concernées
  // (ayant le lot concerné en stock actif)
  const pharmaciesConcernees = await this.prisma.$queryRaw`
    SELECT DISTINCT l.pharmacie_id, array_agg(l.numero_lot) as lots
    FROM lots l
    JOIN medicaments m ON l.medicament_id = m.id
    WHERE l.quantite > 0
    AND m.dci ILIKE ${alerte.dciConcernee}
    AND l.numero_lot = ANY(${alerte.numerosLotConcernes})
    GROUP BY l.pharmacie_id
  `;

  // 3. Identifier les patients concernés
  // (ayant acheté le lot dans les 90 derniers jours)
  const patientsConcernes = await this.getPatientsConcernes(alerte, 90);
```

```
// 4. Diffusion multi-canal aux pharmaciens
for (const pharmacie of pharmaciesConcernees) {
  await this.prisma.diffusionAlerte.create({
    data: { alerteId, pharmacieId: pharmacie.pharmacie_id,
      lotsConcernees: pharmacie.lots,
      canalEnvoi: ['PUSH', 'SMS'] }
  });
  await this.notifService.pushAlerteDPMED(pharmacie.pharmacie_id, alerte);
  await this.smsService.sendAlerteDPMED(pharmacie.pharmacie_id, alerte);
}

// 5. Diffusion aux patients concernés
for (const patient of patientsConcernees) {
  await this.notifService.pushAlertePatient(patient, alerte);
  if (patient.telephone) await this.smsService.sendAlertePatient(patient,
    alerte);
}

// 6. Mise à jour compteurs
await this.prisma.alerteDPMED.update({
  where: { id: alerteId },
  data: {
    statut: 'DIFFUSEE',
    nbOfficinesNotifiees: pharmaciesConcernees.length,
    nbPatientsNotifies: patientsConcernees.length
  }
});
}
```

7.3 Webhook DPMED — Réception sécurisée

```
// Endpoint de réception des alertes depuis le système DPMED
@Post('/webhooks/dpmed')
@UseGuards(DPMEDWebhookGuard) // vérifie signature + IP whitelist DPMED
async receiveDPMEDAlert(@Body() payload: DPMEDAlertPayload) {

  // 1. Vérifier la signature numérique DPMED
  const isValid = this.verifyDPMEDSignature(
    payload, process.env.DPMED_PUBLIC_KEY
  );
  if (!isValid) throw new ForbiddenException('MH-SEC-002: Signature DPMED
    invalide');

  // 2. Créer l'alerte en base
  const alerte = await this.prisma.alerteDPMED.create({ data:
    { ...payload } });

  // 3. Enqueueur immédiatement avec priorité CRITIQUE
  await this.alertesQueue.add('diffuser', { alerteId: alerte.id }, {
    priority: 1,
    delay: 0 // immédiat
  });
}
```

```
    return { received: true, alerteId: alerte.id };  
  }  
  
  // En l'absence d'API DPMED : saisie manuelle via Admin MédiHelm  
  // + confirmation humaine obligatoire avant mise en queue
```

8. Module M19 — Conformité Réglementaire : Spécifications

8.1 Calcul du score de conformité

```

async calculerScoreConformite(pharmacieId: string): Promise<ScoreConformite> {

  // 1. Registre stupéfiants (25 pts)
  //   Vérifie que toutes les délivrances de stupéfiants ont un numéro de
  registre
  const stupMissing = await this.prisma.vente.count({
    where: { pharmacieId,
      lignes: { some: { medicament: { estStupefiant: true } } },
      ordonnanceId: null // sans ordonnance liée
    }
  });
  const scoreStup = stupMissing === 0 ? 25 : Math.max(0, 25 - stupMissing * 5);

  // 2. Alertes DPMED traitées < 24h (25 pts)
  const alertes = await this.prisma.diffusionAlerte.findMany({
    where: { pharmacieId, dateEnvoi: { gte: subDays(new Date(), 90) } }
  });
  const traitees24h = alertes.filter(a =>
    a.dateAcquittement &&
    differenceInHours(a.dateAcquittement, a.dateEnvoi) <= 24
  ).length;
  const scoreAlertes = alertes.length === 0 ? 25
    : Math.round((traitees24h / alertes.length) * 25);

  // 3. Documents valides (20 pts)
  const docsExpires = await this.prisma.documentReglementaire.count({
    where: { pharmacieId, statut: { in: ['EXPIRE', 'EXPIRE_BIENTOT'] } }
  });
  const scoreDocs = docsExpires === 0 ? 20 : Math.max(0, 20 - docsExpires * 5);

  // 4. Signalements EI soumis (15 pts)
  const eiNonSoumis = await this.prisma.signalementEI.count({
    where: { pharmacieId, statutEnvoi: 'EN_ATTENTE',
      createdAt: { lte: subDays(new Date(), 7) } } // > 7j non soumis
  });
  const scoreEI = eiNonSoumis === 0 ? 15 : Math.max(0, 15 - eiNonSoumis * 5);

  // 5. PV destructions à jour (15 pts)
  const destructSansVP = await this.prisma.destructionMedicament.count({
    where: { pharmacieId, pvUrl: null }
  });
  const scoreDestr = destructSansVP === 0 ? 15 : Math.max(0, 15 -
    destructSansVP * 5);

  const scoreTotal = scoreStup + scoreAlertes + scoreDocs + scoreEI +
    scoreDestr;

  return this.prisma.scoreConformite.upsert({
    where: { pharmacieId },

```

```

    create: { pharmacieId, scoreTotal, scoreRegistreStup: scoreStup,
              scoreAlerteDPMED: scoreAlertes, scoreDocuments: scoreDocs,
              scorePharmacovigi: scoreEI, scoreDestructions: scoreDestr },
    update: { scoreTotal, scoreRegistreStup: scoreStup,
              scoreAlerteDPMED: scoreAlertes, scoreDocuments: scoreDocs,
              scorePharmacovigi: scoreEI, scoreDestructions: scoreDestr,
              calculatedAt: new Date() }

  });
}

```

8.2 Génération des exports réglementaires

Export	Format	Librairie	Contenu clé
Registre stupéfiants	PDF signé + Excel	PDFKit + ExcelJS	Date, patient pseudonymisé, médicament, lot, quantité, pharmacien, ordonnance ref
Déclaration trimestrielle	PDF format DPMED	PDFKit	Synthèse stupéfiants + psychotropes par DCI sur le trimestre
PV de destruction	PDF format officiel DPMED	PDFKit	Médicaments, lots, quantités, motif, date, pharmacien responsable, témoin
Rapport pharmacovigilance mensuel	PDF format OMS/DPMED	PDFKit	Signalements EI du mois, gravité, DCI, évolution
Etat de stock annuel	Excel	ExcelJS	Stock par DCI au 31/12, valeur CMUP, péremptions

9. Portail Institutionnel — Spécifications Frontend

9.1 Architecture du portail

Le portail institutionnel est une application Next.js distincte, servie sur `institutionnel.medihelm.com`. Elle est complètement séparée de `app.medihelm.com`. Seuls les rôles `DPMED_ADMIN`, `SOBAPS_VIEWER`, `ABRP_VIEWER` et `GROSSISTE_PARTNER` y ont accès, sur invitation de YEHI OR Tech uniquement.

9.2 Interfaces par acteur

Acteur	Routes	Fonctionnalités
DPMED	/ alertes, /alertes/nouvelle, /alertes/:id, /dashboard, /pharmacovigilance	Emission alertes, suivi taux acquittement, consultation signalements EI anonymisés
SoBAPS	/ livraisons, /livraisons/:id, /dashboard	Confirmations réception, taux conformité, écarts signalés par officines
ABRP	/ dashboard, /tensions, /carte	Agrégats anonymisés : médicaments en tension, carte zones sous-approvisionnées
UbiPharm / Promopharma	/ commandes, /dashboard, /catalogue	Commandes reçues des officines MédiHelm, tendances agrégées de la demande

9.3 Tableau de bord ABRP — Données agrégées anonymisées

```
// Requête agrégée ABRP — aucune donnée individuelle
GET /portail/abrp/dashboard

// Réponse : agrégats uniquement
{
  médicamentsEnTension: [
    { dci: 'Amoxicilline 500mg', nbOfficinesEnRupture: 45, nbOfficinesTotal: 120,
      tauxRupture: 0.375, tendance: 'HAUSSE' }
  ],
  carteApprovisionnement: [
    { departement: 'Littoral', scoreApprovisionnement: 72 },
    { departement: 'Alibori', scoreApprovisionnement: 38 } // alerte sous-approvisionnement
  ],
  top10DciTension: [...],
  evolutionMensuelle: [...],
  // JAMAIS : nom d'officine, stock individuel, données patients, CA
}
```

10. Sécurité Renforcée — mTLS et Données Institutionnelles

10.1 Authentification mutuelle mTLS

```
// mTLS entre MédiHelm API et les systèmes institutionnels
// Configuration NestJS + Node.js TLS

const httpsOptions = {
  key: fs.readFileSync('certs/medihelm-server.key'),
  cert: fs.readFileSync('certs/medihelm-server.crt'),
  ca: fs.readFileSync('certs/dpmed-ca.crt'), // CA du Ministère Santé
  requestCert: true,
  rejectUnauthorized: true // rejette si pas de certificat client valide
};

// Chaque institution dispose de son propre certificat client
// émis et révocable par YEHI OR Tech
// dpmed.medihelm.com.crt
// sobaps.medihelm.com.crt
// ubipharm.medihelm.com.crt
```

10.2 Validation de la signature numérique des alertes DPMED

```
// Vérification signature RSA-256 des alertes DPMED
async verifyDPMEDSignature(payload: any, signature: string): Promise<boolean> {
  const publicKey = await this.keyStore.getDPMEDPublicKey();
  const verify = crypto.createVerify('SHA256');
  verify.update(JSON.stringify(payload));
  return verify.verify(publicKey, signature, 'base64');
}

// En cas d'API DPMED non disponible :
// Alerte saisie manuellement par YEHI OR Tech Admin
// + double validation par 2 membres de l'équipe YEHI OR Tech
// + notification SMS de confirmation envoyée au contact DPMED
// avant mise en queue de diffusion
```

10.3 IP Whitelist — Webhooks institutionnels

```
// Guard NestJS - IP whitelist par institution
@Injectable()
export class InstitutionWebhookGuard implements CanActivate {
  private whitelists = {
    dpmed: process.env.DPMED_IP_WHITELIST?.split(',') || [],
    ubipharm: process.env.UBIPHARM_IP_WHITELIST?.split(',') || [],
    promopharma: process.env.PROMOPHARMA_IP_WHITELIST?.split(',') || [],
    sobaps: process.env.SOBAPS_IP_WHITELIST?.split(',') || [],
  };
};
```

```
canActivate(ctx: ExecutionContext): boolean {  
  const req = ctx.switchToHttp().getRequest();  
  const institution = req.params.institution;  
  const clientIP = req.ip || req.connection.remoteAddress;  
  return this.whitelists[institution]?.includes(clientIP) ?? false;  
}  
}
```

11. Queue de Diffusion d'Alertes — Architecture Haute Priorité

11.1 Garantie de livraison < 2 minutes

Etape	Durée max	Action	Fallback
Réception webhook DPMED	< 1 seconde	Validation signature + création AlerteDPMED	Rejet si signature invalide
Mise en queue BullMQ priorité 1	< 100ms	Job créé avec delay = 0	—
Identification pharmacies concernées	< 10 secondes	Query PostgreSQL sur lots actifs	Index sur lots.numero_lot + médicaments.dci
Identification patients concernés	< 10 secondes	Query sur lignes_vente + commandes_patient	Index sur created_at + medicament_id
Envoi push notifications	< 30 secondes	Firebase FCM batch (jusqu'à 500 tokens/batch)	Retry x3 si FCM error
Envoi SMS	< 60 secondes	AfricasTalking bulk SMS	Retry x5 si timeout
Total garanti	< 2 minutes	Pour jusqu'à 600 pharmacies + 10 000 patients	SLA interne

11.2 Gestion des accusés de réception

```
// Le pharmacien acquitte une alerte depuis MédiHelm Pro
PATCH /alertes/dpmed/:id/acquitter

// Met à jour DiffusionAlerte.dateAcquittement
// Incrémente AlerteDPMED.nbOfficinesAcquittees
// Notifie le portail DPMED via WebSocket (temps réel)

// Si non acquittée après 4h : relance automatique
// Si non acquittée après 24h : escalade — email DIRECTEUR + SMS urgence
// Si non acquittée après 72h : alerte YEHI OR Tech Admin pour contact direct
```

12. Analytics IA v2.0 — Domaines Enrichis

12.1 Nouveau domaine : Conformité réglementaire

```
// Rapport Analytics IA - Domaine CONFORMITE

const rapportConformite = {
  domaine: 'CONFORMITE',
  scoreGlobal: scoreConformite.scoreTotal, // recalculé quotidiennement
  couleur: score >= 75 ? 'VERT' : score >= 50 ? 'AMBER' : 'ROUGE',

  alertes: [
    { type: 'ALERTE_DPMED_NON_TRAITEE', priorite: 'CRITIQUE',
      message: '2 alertes DPMED en attente depuis > 24h', cta:
        '/alertes/dpmed' },
    { type: 'DOCUMENT_EXPIRE', priorite: 'HAUTE',
      message: 'Licence exploitation expire dans 28 jours', cta:
        '/conformite/documents' }
  ],

  recommandations: [
    { titre: 'Soumettre les 3 signalements EI en attente',
      description: 'Ces signalements ont plus de 7 jours sans envoi DPMED.',
      impact: 'ELEVE', urgence: 'CETTE_SEMAINE' }
  ],

  kpis: {
    scoreConformiteTotal: 68,
    alertesDPMEDEnAttente: 2,
    signalementsEIEnAttente: 3,
    documentsExpirantBientot: 1,
    certificationDPMED: false
  }
};
```

12.2 Nouveau domaine : Performance intégration grossiste

```
// Rapport Analytics IA - Domaine INTEGRATION_GROSSISTE

const rapportGrossiste = {
  kpis: {
    commandesViaAPI: 45, // ce mois
    commandesManuellesMois: 3, // anomalies
    delaiMoyenConfirm: '4h', // UbiPharm ce mois
    tauxConfirmation: 0.98, // 98% confirmées
    economiesEstimees: '~45 min/semaine vs commande tél',
    alertesStockAutoGenerees: 12,
    commandesAutoValidees: 10,
    commandesAutoModifiees: 2
  },
  recommandations: [
    { titre: 'Activer commande auto pour Amoxicilline 500mg',
```

```
    description: 'Ce médicament a été en rupture 3 fois ce mois. La commande
    automatique éviterait ces ruptures.',
    impact: 'ELEVÉ', urgence: 'CETTE_SEMAINE' }
  ]
};
```

13. Tests v2.0 — Parcours Institutionnels

13.1 Nouveaux parcours E2E critiques

- Alerte DPMED complète : envoi webhook DPMED > validation signature > identification pharmacies et patients concernés > diffusion push + SMS < 2 min > acquittement pharmacien > mise à jour portail DPMED.
- Commande API grossiste : pharmacien valide commande > transmise API UbiPharm > confirmation webhook UbiPharm > mise à jour statut dans MédiHelm > notification pharmacien.
- Signalement EI : pharmacien remplit formulaire > soumission API DPMED > accusé réception > référence DPMED enregistrée > mise à jour score conformité.
- Réception SoBAPS avec écart : scan bon livraison > saisie quantités réelles ≠ quantités BL > écart détecté > stock mis à jour du reçu > webhook écart envoyé à SoBAPS.
- Score conformité : vérification que le calcul des 5 composantes est correct sur données de test connues.
- Portail DPMED : connexion rôle DPMED_ADMIN > émission alerte > consultation taux acquittement temps réel.
- Sécurité webhook : envoi webhook DPMED avec signature invalide > rejet 403 > aucune alerte créée.

13.2 Tests de sécurité institutionnels

Test	Vecteur d'attaque	Résultat attendu
Fausse alerte DPMED	Webhook sans certificat client valide	HTTP 403 — rejeté avant traitement
Signature DPMED invalide	Payload modifié après signature	HTTP 403 — vérification signature échouée
Accès portail DPMED sans rôle	JWT sans rôle DPMED_ADMIN	HTTP 403 — RolesGuard
Injection données pharmacie dans portail ABRP	Tentative de requête données individuelles	Données agrégées uniquement — schéma séparé
IP non whitelisted sur webhook	IP inconnue sur POST /webhooks/dpmed	HTTP 403 — InstitutionWebhookGuard

14. Déploiement v2.0 — Variables d'Environnement Etendues

14.1 Nouvelles variables d'environnement

Variable	Description	Sensible
DPMED_WEBHOOK_SECRET	Secret HMAC pour webhooks DPMED entrants	Oui
DPMED_PUBLIC_KEY	Clé publique RSA DPMED pour vérification signature alertes	Oui
DPMED_API_URL	URL API DPMED si disponible (sinon null = mode manuel)	Non
DPMED_IP_WHITELIST	IPs autorisées pour webhooks DPMED (CSV)	Oui
SOBAPS_WEBHOOK_SECRET	Secret HMAC webhooks SoBAPS	Oui
SOBAPS_API_URL	URL API SoBAPS	Non
SOBAPS_IP_WHITELIST	IPs autorisées webhooks SoBAPS	Oui
UBIPHARM_API_URL	URL API UbiPharm commandes	Non
UBIPHARM_API_KEY	Clé API UbiPharm (stockée chiffrée)	Oui
UBIPHARM_WEBHOOK_SECRET	Secret HMAC confirmations UbiPharm	Oui
UBIPHARM_IP_WHITELIST	IPs autorisées webhooks UbiPharm	Oui
PROMOPHARMA_API_URL	URL API Promopharma commandes	Non
PROMOPHARMA_API_KEY	Clé API Promopharma (stockée chiffrée)	Oui
PROMOPHARMA_WEBHOOK_SECRET	Secret HMAC confirmations Promopharma	Oui
MTLS_CERT_PATH	Chemin certificat serveur MédiHelm (mTLS)	Oui
MTLS_KEY_PATH	Chemin clé privée serveur (mTLS)	Oui
MTLS_CA_PATH	Chemin CA institutionnelle (mTLS)	Oui
REDIS_ALERTES_QUEUE	URL Redis dédiée queue haute priorité alertes	Oui
INSTITUTIONNEL_SUBDOMAIN	Sous-domaine portail institutionnel	Non

14.2 Checklist déploiement v2.0

- Migrations Prisma : nouveaux schémas qualite, institutionnel, grossistes créés.

- Index PostgreSQL : lots.numero_lot + médicaments.dci pour recherche alertes < 10 sec.
- Certificats mTLS : générés et testés avec les partenaires institutionnels.
- Whitelists IP : configurées par environnement (staging + production).
- Queue BullMQ alertes : Redis dédié configuré avec priorité 1 et concurrence 20.
- Portail institutionnel : déployé sur sous-domaine séparé avec auth distincte.
- Test alerte DPMED de bout en bout sur staging avant tout déploiement production.